Read Deep, Write Shallow: How Looped Language Models Should Spend Their Depth

Yang Hu

Abstract

Looped (recurrent-depth) language models run the same number of loops when reading the prompt and when writing the answer. A small depth-split connector—LoRA plus per-loop key–value maps on a frozen model—lets the prompt be read with p loops and the answer written with d loops. On Ouro-2.6B, the full 4×4 map shows a division of labour: *depth spent reading the prompt buys understanding, while depth spent writing the answer buys multi-step derivation.* At one writing loop, reading 4 loops instead of 1 adds 15.2 MMLU and 8.6 RULER-16K points, and the effect replicates on five more checkpoints. On SenseNova-Loop-9B, whose one-loop writer collapses, merely reading a two-loop prompt state, without any training, cuts unanswered GSM8K problems from 97% to 44%. Writing shallow is where time is saved: a connector writing two loops comes within one point of the 4-loop Ouro-2.6B-Thinking on GSM8K at $0.55\times$ its time per answer. Through a RoPE-commuting key scale, one prompt KV cache serves every loop exactly; storing the answer once too more than doubles long-answer decode throughput. Reading deep hurts short-context retrieval and few-shot mathematics, consistent with linear probes: deeper loops keep less of what was said and more of what comes next.

1 Introduction

Looped language models such as Ouro (Zhu et al., 2025), Huginn (Geiping et al., 2025) and Mixture-of-Recursions (Bae et al., 2025), in the line of Universal Transformers (Dehghani et al., 2018), obtain depth without parameters by running a shared block of layers several times per token. The price is paid at inference: every generated token runs every loop, and a cache per loop multiplies decode memory. Yet the two phases of inference are not alike. Prefill is parallel and paid once per request; decode is sequential and paid per generated token. A looped model nevertheless spends the same depth on both.

Unequal depths have been used before. Continuous Depth Batching runs full-depth prefill and exits early during decode (Schwethelm et al., 2026); OverFill pairs a large prefill model with a small decoder (Kim et al., 2025); Fixed Points Part II varies prefill recurrences at a fixed decode depth (Huang et al., 2026). What has not been measured is *what each depth buys*. The obstacle is practical: a model that writes with fewer loops than it read with must attend to prompt keys and values produced by a deeper loop than its own, and naive sharing breaks exactly where it would be most useful. In our experiments, a trained terminal-KV sharing baseline loses 17.5 GSM8K points at one writing loop (§4.7).

We train one small connector on frozen checkpoints that makes every (read, write) pair up to four usable, and use it to map the depth-allocation space (Figure 1). Our contributions:

- **Read-deep / write-shallow, measured.** A 4×4 read $\times$ write map on Ouro-2.6B shows that reading depth moves knowledge and long-context understanding while writing depth moves multi-step derivation; the read-depth effect replicates on five more checkpoints. Effects of adaptation and of reading depth are reported separately.
- **Shallow-writer wandering, and what deep reading does to it.** One-loop writers of reasoning models often fail to answer (43% of GSM8K answers hit the budget on Ouro-2.6B-Thinking; 97% end without a final answer on SenseNova-Loop-9B). With the writer fixed, reading deeper makes it finish more often and answer more briefly, most on the hardest problems—without any training on SenseNova. Its hesitation density per token does not change; it reaches an answer

sooner.

- **RoPE-commuting KV maps.** A per-frequency complex scale on keys commutes with rotary embeddings (Su et al., 2021), so it can be folded into the query; with a post-attention value map, one stored prompt cache serves every decode loop exactly. Trained in the inference-time (self-live) view, one shared answer bank matches per-loop answer banks at two to three writing loops and, on long answers, more than doubles decode throughput at four (at a 2.6-point GSM8K cost).
- **Shallow KV keeps what was said, deep KV keeps what comes next.** Our failure cases (verbatim retrieval, few-shot mathematics) and the opposite first-loop / last-loop sharing choices of prior work are consistent with one pattern, which linear probes of the cache reproduce and a prompting control supports: on MATH-500, reading deep costs 9.8 points with four exemplars and gains 7.0 without them.

2 Method

Notation. "read p / write d" prefills the prompt with p loops and decodes every answer token with d loops; native d/d is the unmodified model.

Prefill. The frozen model reads the prompt with p loops and keeps only the last loop's prompt keys and values (one bank, RoPE applied).

Decode. The same frozen model, with rank-16 LoRA (Hu et al., 2021) on the attention projections, runs d loops per token. Decode loop k reads the prompt bank through a map indexed by $(k, \text{layer}, \text{KV head})$ (Figure 2):

- *Key map*: one complex scale c_j per rotary frequency pair j. Writing a rotated key pair as a complex number $\tilde{k}_j = e^{im\theta_j} k_j$, scaling commutes with the rotation, $c_j \tilde{k}_j = e^{im\theta_j}(c_j k_j)$, and $\mathrm{Re}\langle q_j, c_j \tilde{k}_j \rangle = \mathrm{Re}\langle \bar{c}_j q_j, \tilde{k}_j \rangle$. The map is therefore applied to the query ($q' = \bar{c} \odot q$) and the stored cache is never rewritten.
- *Value map*: a full $D \times D$ linear map W_k per head. Because attention is linear in the values, $\sum_i a_i W_k v_i = W_k \sum_i a_i v_i$: the map is applied once to the attention output over the prompt.
- Prompt and generated tokens share one softmax: the prompt part and the generated part are computed as two attention calls and merged by their log-sum-exp; only generated tokens keep per-loop KV (their number $t \ll n$).

The prompt KV is thus one loop's worth for every d ("single bank"): total KV is $b(n + d\,t)$ for batch b, prompt length n and t generated tokens, versus $b\,d(n + t)$ for the native model. The single-bank form is algebraically equal to materialising a transformed prompt cache per loop; in BF16, greedy decisions matched on all 512 checked steps.

Training. We minimise $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ over all answer positions, with the native 4/4 model as teacher, on 48M answer tokens of a mixture of reasoning traces, instruction data, long-context data and web text. One (p, d) cell is sampled per micro-batch, so a single set of parameters (63M, 2.4% of the model) serves all 16 cells. The base model stays frozen; LoRA is merged at serving time.

3 Setup

Models. Ouro-2.6B (main, 4 loops), Ouro-1.4B (scale), Nanbeige4.2-3B (second family, 2 loops) and Ouro-2.6B-Thinking and Ouro-1.4B-Thinking (reasoning models, zero-shot with the chat template) (Zhu et al., 2025), and SenseNova-Loop-9B-Preview (third family: a hybrid of Gated-DeltaNet and full-attention layers whose middle block is visited twice; §4.3).

Benchmarks. Full or large sets: GSM8K (Cobbe et al., 2021) (1,319; 3-shot CoT for base models, zero-shot for thinking models), MMLU (Hendrycks et al., 2020) (14,042; 5-shot), RULER (Hsieh et al., 2024) 4K/8K/16K (520 each), HELMET (Yen et al., 2024) (1,108), BBH (Suzgun et al., 2022) (3-shot CoT, 1,620), MMLU-Pro (Wang et al., 2024) (5-shot CoT, 980), ARC-Challenge (Clark et al., 2018) (25-shot, 1,172), MATH-500 (Hendrycks et al., 2021) and HumanEval (Chen et al., 2021). We use lm-evaluation-harness prompts and math-verify scoring; Ouro's native numbers reproduce. All comparisons are paired (same items) with 95% bootstrap intervals.

Comparisons. Three numbers isolate the effects: native d/d; the connector at read d / write d,

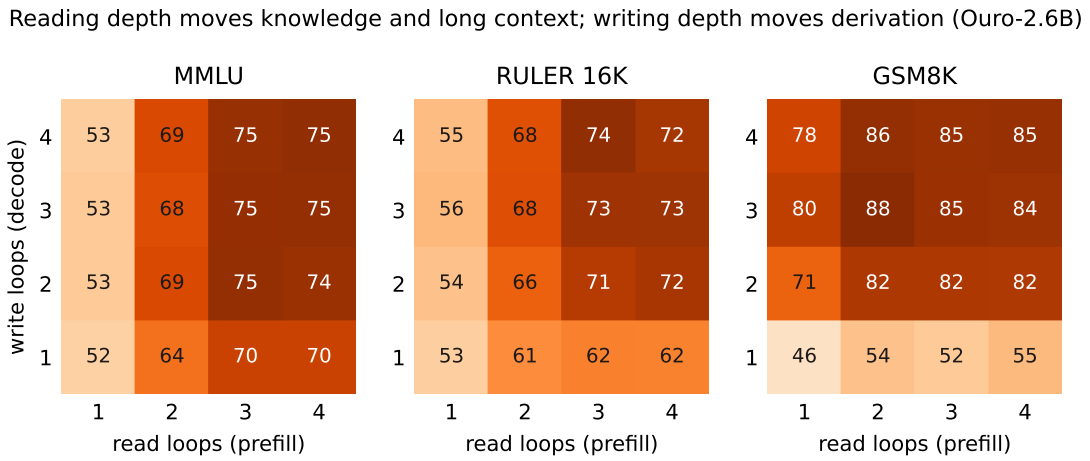


Figure 1: One connector on frozen Ouro-2.6B, all 16 (read, write) pairs, on evaluation subsets used for the grid (MMLU 20 questions per subject, RULER-16K 260 rows, GSM8K 400 problems, 3-shot CoT). Reading depth (columns) moves MMLU and RULER-16K; writing depth (rows) moves GSM8K. Full-set numbers for selected cells are in Tables 1–2.

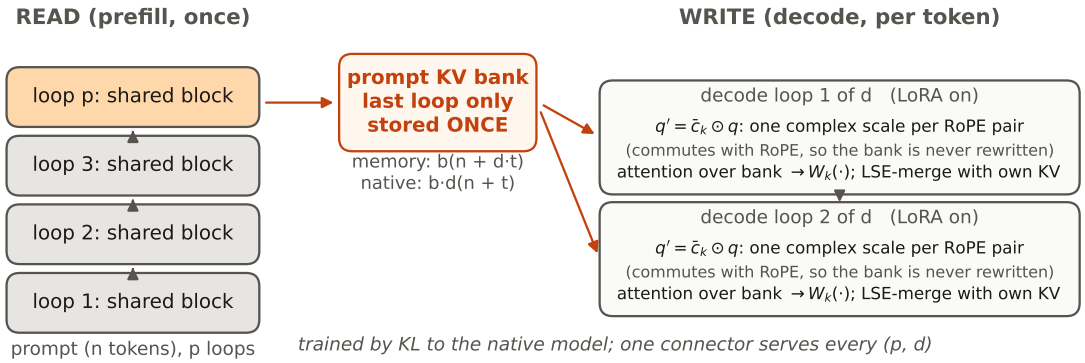


Figure 2: The depth-split connector. The frozen model reads the prompt with p loops and keeps one prompt KV bank (the last loop's). Each of the d decode loops reads that bank through its own map: a per-frequency complex key scale, folded into the query because it commutes with RoPE, and a value map applied after attention; the result is merged by log-sum-exp with attention over the generated tokens' own KV.

which carries the same adaptation; and read p / write d with $p > d$. Only the last two isolate reading depth; against the native model, gains include adaptation.

4 Results

4.1 Reading buys understanding, writing buys derivation

Figure 1 shows the full grid for Ouro-2.6B. MMLU rises from 52–53 at read 1 to 70–75 at read 3–4, independent of writing depth; RULER-16K rises from 53–56 to 62–74. GSM8K barely moves along a row at write 1 (46–55) and rises with writing depth (to 78–88 at write 3–4). Untrained, the frozen model reading deep KV still gains MMLU, but GSM8K and RULER collapse at write 1–2 (read 4 / write 1: GSM8K 13.0, RULER-16K 7.6): the connector is what makes the combination usable.

Table 1 isolates reading depth within the connector. At one writing loop, reading 4 loops instead of 1 adds 15.2 MMLU, 8.0 GSM8K and 8.6 RULER-16K points. The gain shrinks as writing deepens: a deep writer reads its own deep prompt representation anyway. On RULER, the read-deep gain grows with prompt length—negative at 4K, positive at 16K—and most of it comes from the second prefill loop (RULER-16K at write 1: 54.3 / 61.6 / 61.3 / 62.8 for read 1–4).

Table 2 extends the comparison against the native model to five more benchmarks. At one writing loop, reading deep adds 15–23 points on BBH,

write d	MMLU	GSM8K	RULER-16K
1	+15.2 [14.3, 16.1]	+8.0 [5.4, 10.5]	+8.6 [4.2, 12.9]
2	+6.0 [5.3, 6.7]	−1.7 [-3.5, 0.1]	+5.3 [1.2, 9.3]
3	+0.6 [0.2, 1.1]	−0.8 [-2.4, 0.8]	−0.4 [-3.0, 2.2]

Table 1: Reading depth at equal writing depth (Ouro-2.6B, one connector): read 4 / write d minus read d / write d, full sets. RULER 4K at write 1: −3.8 [-7.8, 0.3].

d	BBH	MMLU-Pro	ARC-C	HELMET	MATH
1	+20.9	+21.4	+14.8	+23.3	−6.0
2	+4.6	+7.8	+2.9	+28.0	−2.6
3	−1.1	–	+0.3	+5.0	−2.2

Table 2: read 4 / write d minus native d/d, Ouro-2.6B. Write 1 intervals: BBH [18.0, 23.9], MMLU-Pro [18.2, 24.7], ARC-C [12.1, 17.3], HELMET [20.3, 26.5], MATH [-10.2, -2.2].

MMLU-Pro, ARC-C and HELMET; at write 2, HELMET reaches the native 4/4 level (53.0 vs. 54.8). MATH-500 is the exception (§6).

Generalisation. The read-depth effect holds across scale, family and training stage (Table 3). On Nanbeige4.2-3B (two loops) the comparison uses separately trained per-cell connectors, so reading depth and connector weights change together; a single universal connector for Nanbeige suffers interference between read depths (§6).

4.2 Shallow writers run out of budget; deep reading helps them finish

A reasoning model written with one loop wanders. On Ouro-2.6B-Thinking, native 1/1 hits the 4,096-token budget on 43.5% of GSM8K problems (almost every unfinished answer is a budget hit, not an early stop) and uses 15.7 hesitation markers ("wait", "alternatively", "re-read") per 1,000 tokens (Table 4). Training the connector at read 1 / write 1 halves the budget hits—an adaptation effect. With the writer fixed, reading 4 loops instead of 1 cuts them further (19.8% → 13.0%), shortens answers by 20% and adds 6.6 accuracy points (on Ouro-1.4B-Thinking: +13.6 [10.4, 16.5], answers without \boxed{} 37.7% → 26.0%), and the gain grows with problem difficulty (Figure 3): on problems whose reference solution has five or more steps, read 4 / write 1 gains 10 points over read 1 / write 1. Hesitation *density* does not fall (12.2 vs. 13.8 per 1,000 tokens): the deep-reading writer does not hesitate less per step, it reaches an answer in fewer steps. At two writing loops the increment is small on GSM8K (read 2 / write 2 92.8 vs. read 4 / write 2 93.3; 858 vs. 836 tokens), and both track native 4/4.

model	read 4 / write 1 − native 1/1 (unless noted)
Ouro-1.4B	RULER-16K +14.3, MMLU +22.4, GSM8K +14.9, ARC-C +17.2; BBH −11.7 (collapse, §6)
Nanbeige4.2-3B	read 2 / write 1 − read 1 / write 1: RULER 4K/8K/16K +11.8/ + 11.7/ + 11.0, MMLU +12.2, GSM8K +5.2, ARC-C +11.6
Ouro-2.6B-Thinking	GSM8K +18.0 [15.2, 20.6], MATH-500 +13.2 [8.5, 18.2], HELMET‡ +25.6 [22.3, 28.9], MMLU-Pro +16.2, ARC-C +14.3
Ouro-1.4B-Thinking	GSM8K +20.1 [17.1, 23.4], MMLU-Pro +15.5 [10.0, 20.7]; within the connector read 4 / write 1 − read 1 / write 1: GSM8K +13.6 [10.4, 16.5], MMLU-Pro +17.1 [12.4, 22.1], HELMET‡ +11.9 [8.9, 14.6]; BBH collapses (§6)

Table 3: Same comparison across models (full sets except: BBH 60 per subtask, HELMET 1,108 rows; Nanbeige RULER-16K 385 paired rows; Thinking MMLU-Pro 420 (both sizes), MATH-500 400). ‡Thinking models on raw HELMET prompts are decoded so that an answer cannot be empty (§6). All positive entries are significant.

config	acc.	budget hit	tokens	marks/1k
native 1/1	59.7	43.5%	2,286	15.7
read 1 / write 1	71.1	19.8%	1,392	12.2
read 4 / write 1	77.7	13.0%	1,113	13.8
native 2/2	92.2	6.3%	957	7.8
read 2 / write 2†	92.8	4.4%	858	5.8
read 4 / write 2	93.3	3.9%	836	5.8
native 4/4	94.2	3.2%	816	5.2

Table 4: Ouro-2.6B-Thinking, GSM8K zero-shot (1,319; †989). Budget hit: answer reached 4,096 tokens. Marks: "wait / alternatively / re-read" per 1,000 generated tokens.

4.3 A third family: SenseNova-Loop-9B, with and without training

SenseNova-Loop-9B-Preview is a 9B hybrid of Gated-DeltaNet and full-attention layers whose middle 24 of 32 layers are visited twice. Its one-visit writer collapses (native 1/1: 2.2% GSM8K, 96.8% of answers without a final \boxed{} answer, 86 "wait / re-read / alternatively" per answer), which makes it a clean test (Table 5).

Without any training. Handing the same one-visit writer the two-visit prefill state—full-attention KV and the Gated-DeltaNet convolution

config	GSM8K	unfin.	R-4K	R-8K	R-16K	HELMET
native 1/1	2.2	96.8	21.6	16.5	14.0	12.3
read 2 / write 1, no training	10.9	43.7	83.5	80.8	80.6	69.5
read 1 / write 1 + connector	95.6	–	85.8	83.3	81.5	60.9
read 2 / write 1 + connector	95.2	–	82.1	81.4	82.7	67.6
native 2/2	95.8	0.3	86.9	85.8	86.1	71.8

Table 5: SenseNova-Loop-9B-Preview. GSM8K zero-shot chat (1,319); RULER with a non-thinking prefill (520 per length); HELMET (6,646); MMLU-Pro 5-shot CoT (980).

config	MMLU-Pro	acc.	tok./ans.	ms/tok.	s/ans.
native 1/1	18.9	2.5	2,500	34.6	86.5
read 4 / write 1	55.7	73.4	1,310	41.9	55.1
native 2/2	60.9	95.3	1,044	66.9	70.0
read 2 / write 2	62.8	96.9	755	82.7	62.6
read 4 / write 2	63.2	96.9	741	84.9	63.1
native 4/4		93.8	879	129.6	114.2

Table 6: End to end per answer (Ouro-2.6B-Thinking, GSM8K, 64 problems, batch 1, one B300, prefill included, real greedy answers). On all 1,319 problems: read 4 / write 2 93.3, native 4/4 94.2, paired -1.0 [-2.0, 0.1].

and recurrent states of the last visit, no parameter changed—cuts unanswered GSM8K problems (no \boxed{}) to 43.7% (-53.1 [-55.7, -50.3]), raises RULER-4K from 21.6 to 83.5 and MMLU-Pro from 18.9 to 55.1 ($+36.2$ [32.8, 39.7]). The writer, its depth and its weights are identical; only what it reads differs. The intervention transfers the whole state—attention KV and Gated-DeltaNet state—so it does not separate the two; RULER uses a non-thinking prefill for every configuration, because the model otherwise spends the 64-token budget thinking.

With a connector. An 11M-parameter LoRA connector (20M KL tokens; Appendix A) trained for a single one-visit decoder that reads either a one- or a two-visit prompt state removes the collapse, an adaptation effect: GSM8K reaches the native 2/2 level whichever depth it reads, and end to end read 1 / write 1 takes $0.59\times$ [0.52, 0.68] the time of native 2/2 and read 2 / write 1 $0.64\times$ [0.50, 0.83] (64 problems, batch 1; 74 and 80 vs. 131 ms per token, close to 32 vs. 56 layer executions). Reading depth then shows where the thesis predicts: with the same decoder, reading two visits beats reading one on long-context HELMET by 6.7 [5.8, 7.7] points, on MMLU-Pro by 5.5 [3.0, 8.1] and on 3-shot BBH by 7.0 [5.9, 8.0] (BBH: all 6,511 problems; native 2/2 57.6, native 1/1 2.6), while on RULER it costs 3.7 points at 4K and is neutral at 16K ($+1.2$ [-1.7, 4.0]), as on Ouro at short contexts.

End to end. Shallow writing through the connector pays end to end (Table 6, Figure 4). read 4 / write 2 answers in $0.55\times$ the time of native 4/4 and $0.90\times$ that of native 2/2. We state the cost plainly: at batch 1, single-bank decoding is $1.27\times$ slower per token than native 2/2 (two attention calls and a merge per layer); the answer arrives sooner because it is 29% shorter. On all 1,319 problems read 4 / write 2 is within one point of native 4/4 (93.3 vs. 94.2) and above native 2/2 (92.2; $+1.1$ [-0.2, 2.4]). The saving over native 4/4 comes from writing two loops through the connector, not from reading deeper: read 2 / write 2 is as fast and as accurate on the same problems (read 4 / write 2 takes $1.01\times$ [0.91, 1.11] the time of read 2 / write 2; read 2 / write 2 takes $0.55\times$ [0.46, 0.64] that of native 4/4). The same holds on MATH-500 (40 problems, 8,192-token budget): read 4 / write 2 matches native 4/4 (85.0 vs. 82.5%) at $0.63\times$ [0.53, 0.77] its time, but is $1.22\times$ [1.04, 1.44] slower than native 2/2 at equal accuracy, because its MATH solutions are not shorter. At write 2, reading deeper pays on long-context reading and knowledge instead (read 4 / write 2 minus read 2 / write 2: HELMET $+15.8$ [13.1, 18.5], ARC-C $+2.3$, BBH $+2.2$; on HELMET read 4 / write 2 is 26.0 [22.9, 29.1] points above native 2/2 and 4.3 [2.5, 6.2] above native 4/4).

4.4 Serving one prompt bank

With a static prompt cache and the same fused attention on both sides (FlashAttention-2 (Dao, 2023) calls merged by log-sum-exp), the best decode throughput over batch sizes at 8K context is $0.97\times$, $1.19\times$, $1.36\times$ and $1.62\times$ that of the native model at writing depths 1–4, because the single bank admits larger batches as d grows (max batch 64 vs. 80/40/24/20). Against the quality reference native 4/4, read 4 / write 1 decodes $6.5\times$ faster (813 vs. 125 tokens/s). These ratios are against the fastest of three native implementations we measured: static per-loop banks with the same two-call flash attention as ours (834 / 342 / 125 tokens/s at $d = 1/2/4$), ahead of the standard dynamic cache (500 / 245 / 105) and of a preallocated contiguous cache with one attention call per layer (267 / 130 / 57, slower because the model's attention mask keeps it off the flash kernel); all variants produce

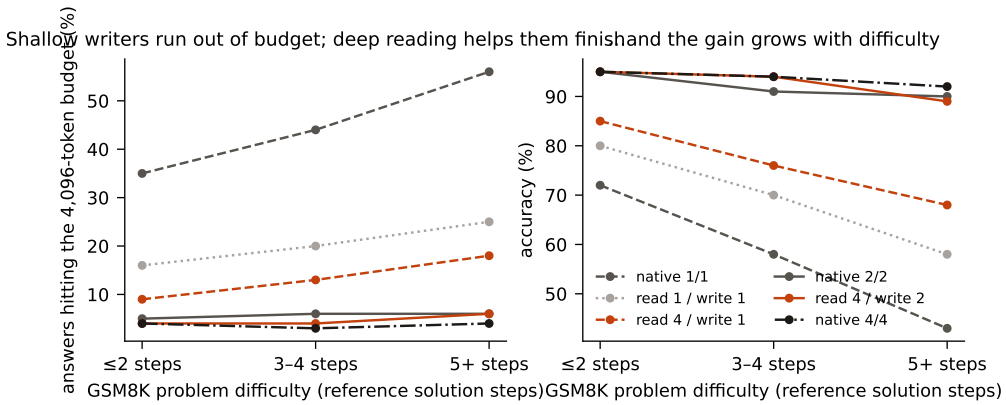


Figure 3: Ouro-2.6B-Thinking on all 1,319 GSM8K problems, bucketed by reference-solution steps.

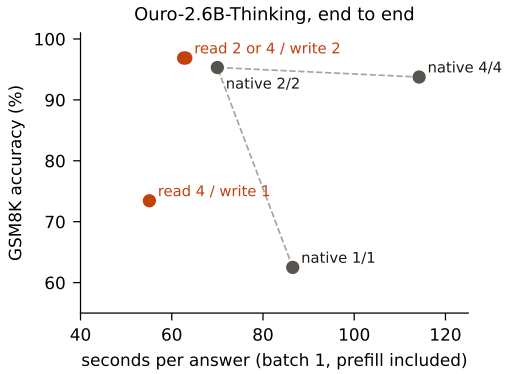


Figure 4: Accuracy vs. time per answer (Table 6). Dashed: native models. Reading 2 or 4 loops before writing 2 gives the same point.

identical greedy tokens. All serving numbers are decode-only; end-to-end cost is in Table 6.

4.5 Storing the answer once, too

The single bank stores the prompt once, but generated tokens still keep one KV bank per writing loop: $b(n + dt)$. Sharing one answer bank across loops is what BeNice! (Prairie et al., 2026) trains for; its training lets each token read its own last-loop KV on the diagonal, which does not exist yet at inference. We train the connector directly in the inference-time view instead (*self-live*[1]): each writing loop reads the prompt, a single answer history written by the last loop, strictly before the current token, and on the diagonal only the KV computed live in the current loop; the history comes from a no-grad pass of the same model (Appendix A). The answer is then stored once. Against the same connector with per-loop answer banks, no difference is significant at two and three writing loops on GSM8K (read 4 / write 2: $+0.3$ [-1.5, 2.2]; read 4 / write 3: $+0.5$ [-0.9, 2.1]; read 3 / write 3: -0.8 [-2.4, 0.8]), MMLU (-0.1 [-0.4, 0.2]) or RULER-4K (four cells); the cost appears as more loops share one history: MATH at three writing loops -3.4 [-6.8, -0.2], GSM8K at four -2.6 [-4.1, -1.0] (and -4.8 against native 4/4). Our evaluation of this arm materialises the per-loop mapped prompt banks; serving with one prompt bank and one answer bank (total KV $b(n + t)$, that of a one-loop model) gives the same greedy tokens (512/512 checked steps). Where answers are long (1K-token prompts, 2,048 generated tokens), it raises the largest batch that fits from 32 (native 4/4) and 64 (one prompt bank, per-loop answer banks) to 192, and decode throughput at four writing loops to $2.17\times$ native 4/4 (476 vs. 219 tokens/s; $1.30\times$ native 2/2 at two loops, where accuracy is unchanged; at four loops the throughput comes with the GSM8K cost above).

4.6 What each loop stores

Why does deep KV help knowledge and long-context reading but hurt verbatim recall (§6)? We fit linear probes to the frozen native model's per-loop *value* cache—what attention reads back from the prompt—at the same layer and with the same weights, varying only the loop (Figure 5). Averaged over probed layers, the previous token is less linearly decodable from loops 2–4 than from loop 1 (Ouro-2.6B: 57.7% $\rightarrow$ 45.5%, lower in 12 of 13 probed layers), and the next token more (26.0% $\rightarrow$ 36.1%, higher in all 13); Ouro-1.4B and Ouro-2.6B-Thinking agree. In the hybrid SenseNova-Loop-9B (six full-attention layers in the looped

[1]Self-live was developed by the author in separate work on shared KV caches for looped models; here it is applied to the connector's answer tokens.

block, visit 1 vs. 2) the next-token gain replicates in every layer ($24.1\% \rightarrow 29.1\%$), but the previous token is about as decodable at both visits on average (56.8% vs. 56.5%; -7.0 at the first of these layers, $+2$ to $+3$ at the last); only attention values are probed, not the Gated-DeltaNet state. The literal loss happens at the second loop, as does the behavioural one: for few-shot MATH, reading 2 or 3 loops is no better than 4; and the behavioural loss disappears, and turns into a gain ($+7.0$), when the exemplars that must be imitated are removed.

These are decodability results, not proof of information loss or causal use (Merin, 2026), and next-token prediction is only a proxy for meaning. Not every layer follows the mean (at layer 0, the previous token becomes more decodable after loop 1). The trend is the across-loop analogue of the across-layer one of Voita et al. (2019); per-loop readouts of Ouro's residual stream show a related latent-prediction trend (Olszowiec et al., 2026), and per-loop probes go back to Yang et al. (2023). What is specific here is the matched-layer comparison of the stored cache: in a looped model, choosing which loop's KV to keep is choosing what to remember.

4.7 Ablations

Plain sharing. A trained terminal-KV sharing baseline in the style of BeNice! (Prairie et al., 2026) (the decoder reads the prefill's last-loop KV with LoRA but no map) loses 17.5 [-20.2, -14.8] GSM8K points at write 1 against our connector, and ties it at write 2 (-0.1 [-2.0, 1.7]). The map is what makes one-loop writing over deep KV work; from two writing loops on, plain sharing suffices.

Giving the writer the first loop back. If short-context recall suffers because deep KV keeps less of what was said, handing the writer the *first* prefill loop's KV should help. A gated attention branch over the first loop's prompt KV (initialised closed; trained like the connector) raises RULER-4K at read 4 / write 1 by 2.3 [1.0, 3.6], recovering about 60% of the read-deep loss, and changes nothing else (GSM8K $+0.7$ [-0.8, 2.2], MMLU $+0.0$; other cells within ± 0.6 on RULER-4K, two of them on 406 and 334 of 520 rows). It does not help at two writing loops, and it does not remove the 4-shot MATH loss (22.2 vs. 32.2, as without the branch). We report this without a causal reading: the branch adds a bank and parameters as well as shallow information.

A second training seed. Retraining the universal connector with another seed (initialisation and cell sampling) reproduces every read-depth effect on full sets, e.g. at one writing loop MMLU $+15.2$ and RULER-16K $+8.6$ under both seeds (Appendix C).

One bank vs. one per loop. Storing a separate mapped bank per decode loop is equal cell by cell (RULER intervals include zero), so the single bank costs nothing.

5 Related Work

Looped and elastic depth. Accordion (Zhang et al., 2026) and Elastic Latent Recurrence (Ladhak et al., 2026) serve several depth budgets from one model trained for it; ELR reports that keeping only the last loop's KV retains most of the gain, the from-scratch counterpart of our single bank. The Parallel Loop Transformer (Wu et al., 2025a), LoopCoder-v2 (Yang et al., 2026) and shared-memory looped transformers (Monea et al., 2026) instead share the *first* loop's KV. None decouple prefill from decode depth; their opposite choices bracket our finding that deep KV serves knowledge and shallow KV serves recall.

Unequal prefill and decode compute. CDB (Schwethelm et al., 2026), OverFill (Kim et al., 2025), Position-Aware Depth Decay Decoding (Fan et al., 2025) and Fixed Points Part II (Huang et al., 2026) use deeper prefill than decode; POP (He et al., 2026) and SPEED (Oh et al., 2026) use the reverse; Decode-Branch Transformers (Liu et al., 2026), PHD (Wu et al., 2025b) and State-Prediction Separation (Langford et al., 2026) add decode-side computation. We contribute a measurement of what each side buys and a connector that makes the full grid usable on existing looped checkpoints.

Cross-loop and cross-model KV. Looped Latent Attention (Neill and Reid, 2026) compresses KV across loops, BeNice! (Prairie et al., 2026) and LSTM-style looped transformers (Kirsch, 2026) share it; Cache-to-Cache (Fu et al., 2025), HeteroFold (Yun et al., 2026), Swarm Telepathy (Alabdulkareem et al., 2026) and CacheLink (Qiu et al., 2026) transfer KV between models, and BellowsKV (Zhang and Chen, 2026) shares a latent across layers. Like CacheLink, we train maps through the receiver's output distribution, never on KV reconstruction; within one model, tokens,

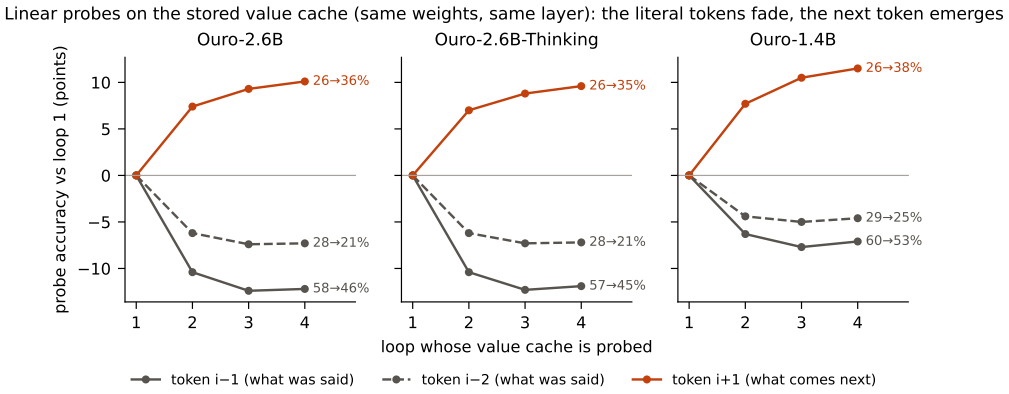


Figure 5: Linear probes on the per-loop value cache of three frozen models, change from loop 1 (mean over 13 / 13 / 9 layers; 384 FineWeb windows of 1,024 tokens, 20% held out; 2,048 most frequent training tokens).

layers and positions align, so a RoPE-commuting scale and a value map suffice and are exact to serve.

6 Conclusion

Depth in a looped model is not one budget but two. Reading depth buys understanding and is paid once; writing depth buys derivation and is paid per token. A small connector makes the two separable on existing checkpoints, a RoPE-commuting map lets one prompt cache serve every writing depth, and reading deeper lets a shallow writer reason with less wandering and in less time.

Limitations

Short-context recall and few-shot mathematics prefer shallow KV. Within the connector at one writing loop, reading 4 loops instead of 1 lowers RULER-4K (-3.8 [-7.8, 0.3]) and 4-shot MATH-500 on the base model (-9.8 [-13.8, -5.8]; -11.2 [-15.6, -7.0] with the same full-answer scoring as below); RULER at 16K moves the other way. On MATH the sign depends on prompting: with the same connector and problems but no exemplars, reading deep *helps* ($+7.0$ [3.2, 11.0]; 40.8 vs. 33.8), and the reversal replicates at two writing loops (read 4 vs. 2: $+7.0$ [3.0, 10.8]) and on Ouro-1.4B (-4.0 [-8.0, 0.0] with exemplars, $+4.8$ [1.2, 8.2] without). Removing exemplars also changes length and format, so we read this as a prompting interaction consistent with, not proof of, deep KV keeping less of what was said (§4.6). Reading fewer loops avoids the loss when exemplars are given.

Deep writing over one deep bank loses retrieval. At three or more writing loops, the single-bank connector trails native d/d on RULER (write 3: $-10.4/-9.5/-5.3$ at 4K/8K/16K), because the native model reads a separate prompt bank per loop.

Deeper prefill costs time to first token. Reading 4 loops costs a 4-loop prefill: in an interim batch-1 measurement on RULER-16K (64-token answers), a read 4 / write 1 request took about $1.6\times$ native 1/1 and half of native 4/4. Deep reading pays for itself only when answers are long or quality requires it.

Transfer is not free. On Nanbeige4.2-3B one universal connector suffers interference between read depths; per-depth adapters fix RULER and MMLU, and doubling the data halves the remaining GSM8K gap.

Collapse at the extreme cell. Both 1.4B checkpoints degenerate at read 4 / write 1 on 3-shot CoT BBH (Ouro-1.4B: empty or echoed answers; Ouro-1.4B-Thinking: 28% empty and -20.5 points against read 1 / write 1); read 1–2, and reading 4 loops at two writing loops ($+8.8$ [6.2, 11.5] over read 2 / write 2), do not.

Empty answers on raw prompts. Thinking checkpoints given raw (non-chat) HELMET prompts often stop at once (a newline, then end of sequence): 42.7% of native 2/2 answers on Ouro-2.6B-Thinking, at most 1.1% for read-4 connectors. For thinking models we therefore mask the stop tokens until an answer has a non-whitespace token, for every configuration. This does not rescue native 2/2 (34.1 vs. 31.4): its empty answers hid no correct ones. Base models are unaffected ($\leq$2.7% empty).

Wandering measures. "Unfinished" means the answer hit the generation budget; hesitation markers are lexical. Neither measures reasoning quality directly.

Scope. Six looped checkpoints from 1.4B to 9B parameters (the full 4×4 grid only on Ouro-2.6B), greedy decoding, English benchmarks; end-to-end latency at batch 1 on 64 problems and one GPU type. Gains against native models include adaptation; reading-depth effects are isolated within the connector, or without training (SenseNova).

References

Abdulrahman Alabdulkareem, Irene Tenison, Yang Zhang, Yada Zhu, Boris Katz, and Dan Gutfreund. 2026. Swarm Telepathy: Prefill-Free KV-Cache Transfer Across Families. *arXiv preprint arXiv:2610.15095*.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. 2025. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. *arXiv preprint arXiv:2507.10524*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.

Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2018. Universal Transformers. *arXiv preprint arXiv:1807.03819*.

Siqi Fan, Xuezhi Fang, Xingrun Xing, Peng Han, Shuo Shang, and Yequan Wang. 2025. Position-Aware Depth Decay Decoding (D^3): Boosting Large Language Model Inference Efficiency. *arXiv preprint arXiv:2503.08524*.

Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. 2025. Cache-to-Cache: Direct Semantic Communication Between Large Language Models. *arXiv preprint arXiv:2510.03215*.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. *arXiv preprint arXiv:2502.05171*.

Junhui He, Zhihui Fu, Jun Wang, and Qingan Li. 2026. POP: Prefill-Only Pruning for Efficient Large Model Inference. *arXiv preprint arXiv:2602.03295*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring Massive Multitask Language Understanding. *arXiv preprint arXiv:2009.03300*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *arXiv preprint arXiv:2103.03874*.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*.

Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. 2026. Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. *arXiv preprint arXiv:2610.06833*.

Woojeong Kim, Junxiong Wang, Jing Nathan Yan, Mohamed Abdelfattah, and Alexander M. Rush. 2025. OverFill: Two-Stage Models for Efficient Language Model Decoding. *arXiv preprint arXiv:2508.08446*.

Andreas Kirsch. 2026. LSTM-Style Looped Transformers with Shared KV Caches. *arXiv preprint arXiv:2610.33360*.

Faisal Ladhak, Shantanu Acharya, Santiago Akle Serano, Simeng Sun, Boris Ginsburg, and Krishna C Puvvada. 2026. One Model to Loop Them All: Elastic Latent Recurrence for Language Models. *arXiv preprint arXiv:2610.12582*.

John Langford, Nathan Godey, Giovanni Monea, Yoav Artzi, Harry Dong, Ying Fan, Gustavo de Rosa, and Zheng Zhan. 2026. Almost Free State Prediction Separation. *arXiv preprint arXiv:2609.03807*.

Liming Liu, Mingze Wang, and Tuo Zhao. 2026. Decode-Branch Transformers: Decoupling the Primary Prefill Path from Additional Decode Computation. *arXiv preprint arXiv:2608.12385*.

Aur Shalev Merin. 2026. Present but Hard to Read: Probing Latent Reasoning with Minimal Pairs. *arXiv preprint arXiv:2610.28498*.

Giovanni Monea, Keshav Ramji, Yousef El-Kurdi, Luis A. Lastras, Yoav Artzi, Nathan Godey, and Ramón Fernandez Astudillo. 2026. The Surprising Effectiveness of Shared Memory in Looped Transformers. *arXiv preprint arXiv:2610.02383*.

James O' Neill and Fergal Reid. 2026. Looped Latent Attention: Cross-Loop KV Compression for Looped Transformers. *arXiv preprint arXiv:2607.15456*.

Jungsuk Oh, Hyeseo Jeon, Hyunjune Ji, Kyongmin Kong, and Jay-Yoon Lee. 2026. Shallow Prefill, Deep Decoding: Efficient Long-Context Inference via Layer-Asymmetric KV Visibility. *arXiv preprint arXiv:2605.06105*.

Pawel Olszowiec, Paulina Tomaszewska, Maciej Żelaszczyk, Michal Byra, and Neo Christopher Chung. 2026. Monitoring latent reasoning in looped language models. *arXiv preprint arXiv:2610.18127*.

Hayden Prairie, Jacob M. Ryan, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. 2026. BeNice! Closing Looping's Inference-Memory Gap via KV-Sharing. *arXiv preprint arXiv:2610.31633*.

Haomiao Qiu, Yining Tang, Zijia Song, Ruizi Han, Ziyue Qiao, and Miao Zhang. 2026. CacheLink: Functional KV Transport for Zero-Prefill Cross-Model Context Handoff. *arXiv preprint arXiv:2610.15509*.

Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. 2026. Depth-adaptive Inference of Looped Language Models via Continuous Depth Batching. *arXiv preprint arXiv:2608.09444*.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864*.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. 2022. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. *arXiv preprint arXiv:2210.09261*.

Elena Voita, Rico Sennrich, and Ivan Titov. 2019. The Bottom-up Evolution of Representations in the Transformer: A Study with Machine Translation and Language Modeling Objectives. In *Proceedings of EMNLP-IJCNLP 2019*, pages 4396–4406.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. *arXiv preprint arXiv:2406.01574*.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. 2025a. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2510.24824*.

Bohong Wu, Shen Yan, Sijun Zhang, Jianqiao Lu, Yutao Zeng, Ya Wang, and Xun Zhou. 2025b. Efficient Pretraining Length Scaling. *arXiv preprint arXiv:2504.14992*.

Jian Yang, Shawn Guo, Wei Zhang, Tianyu Zheng, Yaxin Du, Haau-Sing Li, Jiajun Wu, Yue Song, Yan Xing, Qingsong Cai, Zelong Huang, Chuan Hao, Ran Tao, Xianglong Liu, Wayne Xin Zhao, Mingjie Tang, Weifeng Lv, Ming Zhou, and Bryan Dai. 2026. LoopCoder-v2: Only Loop Once for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2606.18023*.

Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. 2023. Looped Transformers are Better at Learning Learning Algorithms. *arXiv preprint arXiv:2311.12424*.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. 2024. HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly. *arXiv preprint arXiv:2410.02694*.

Vincent-Daniel Yun, Woosang Lim, Haneul Yoo, Sungjoo Yoo, Murali Annavaram, and Sai Praneeth Karimireddy. 2026. Prefill-Free Cross-Family KV Cache Transfer for Heterogeneous Multi-Agent LLMs. *arXiv preprint arXiv:2609.32259*.

Jiamu Zhang and Hanjie Chen. 2026. BellowsKV: One Latent for Many Layers, Expanded on Demand. *arXiv preprint arXiv:2610.35023*.

Yudong Zhang, Wei Ding, Kaili Zheng, Yuchen Ding, and Zhenyu Zhang. 2026. Accordion: Elastic-Depth Inference via Fixed-Prefix Layer Looping. *arXiv preprint arXiv:2610.32492*.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, and 14 others. 2025. Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*.

A Training Details

Connector. Rank-16 LoRA ($\alpha = 16$, no dropout) on the query, key, value and output projections, active only in decode; per (decode loop, layer, KV head) a complex scale for each rotary frequency pair on keys and a full $D \times D$ value map, both initialised to the identity. Trainable parameters: 63,307,776 for Ouro-2.6B (2.4%). The backbone is frozen.

Objective. $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ summed over answer positions and normalised by the number of KL tokens in the global batch; the teacher is the native 4/4 model with the same weights. One cell (p, d) is sampled per micro-batch from all 16 pairs.

Data. 48M KL tokens: OpenThoughts reasoning traces 45%, Tulu SFT responses 40%, LongAlpaca long-context answers 5% and FineWeb-Edu windows 10% (fractions of KL tokens); prompts up to 7,680 tokens, answers up to 1,024. Base models see a plain "User: … Assistant:" format; thinking models see the same samples in their chat template. No benchmark data enters training.

Optimisation. AdamW ($\beta = (0.9, 0.95)$, no weight decay), peak learning rate 10^{-4} with 2% linear warm-up and cosine decay to 10^{-5}, gradient clipping at 1.0, global batch 24 sequences on 4 GPUs, bfloat16 backbone.

SenseNova-Loop-9B. Rank-16 LoRA on the attention projections of the looped layers (full attention: q/k/v/o; Gated-DeltaNet: input and output projections), active only in decode; no maps. The one-visit decoder reads the prefill state of the last of $p \in \{1, 2\}$ visits (sampled 50/50), so the same decoder serves read 1 and read 2. Teacher: native 2/2. Data: the same mixture re-rendered in the SenseNova chat template (60,000 samples), 20M KL tokens, peak learning rate 2×10^{-4}.

Self-live answer bank. Generated tokens keep one KV bank per layer shared by all writing loops; at inference the last writing loop commits each token to it. Training uses the same KL objective as the connector, computed through the inference-time view in two passes: a no-grad pass in which each loop reads its own causal answer KV records the last loop's answer KV; a second pass reads, at every loop, the prompt bank, that recorded history strictly before the current position, and the current token's live KV on the diagonal. Prompt side, data and budget as for the connector.

B Evaluation Details

All tasks run through lm-evaluation-harness with greedy decoding and our runner. Base models: GSM8K 3-shot CoT (1,319), MMLU 5-shot (14,042), BBH 3-shot CoT (60 per subtask, 1,620), MMLU-Pro 5-shot CoT (70 per subject, 980), ARC-Challenge 25-shot (1,172), MATH-500 4-shot Minerva, HumanEval. Long context: RULER 4K/8K/16K (520 rows each, 13 tasks $\times$ 40) and HELMET (1,108). Thinking models: zero-shot with the chat template, answers in `\boxed{}` scored by math-verify, budget 4,096 tokens (GSM8K) and 8,192 (MATH-500). MATH-500 is scored with math-verify throughout; Minerva's strict "Final Answer" match would reverse the 4-shot read-depth comparison on Ouro-2.6B (19.6 vs. 9.8) because the read-1 writer drops the required format more often, which is why we do not use it. Intervals are paired bootstrap 95% over items (2,000 or 5,000 resamples); for HELMET, resampling clusters equals resampling rows.

C Training-Seed Robustness

comparison	task	seed 1	seed
read 4 / write 1 − read 1 / write 1	GSM8K	+8.0 [5.5, 10.3]	+7.2 [4.5, 9
	MMLU	+15.2 [14.3, 16.1]	+15.2 [14.4, 16
	RULER-4K	−3.8	−3
	RULER-8K	+2.5	+2
	RULER-16K	+8.6 [4.5, 12.5]	+8.6 [4.1, 12
read 4 / write 2 − read 2 / write 2	GSM8K	−1.7 [-3.5, 0.2]	−2.9 [-4.8, -1
	RULER-4K	−7.2	−6

Table 7: Ouro-2.6B universal connector trained with two seeds; paired differences within each seed on full sets.

D Compute

The universal Ouro-2.6B connector trains in 6.7 hours on 4 B300 GPUs (48M KL tokens). Evaluations, probes and timing used two 8-GPU B300 nodes; end-to-end timing uses one otherwise idle GPU per configuration.